AgriAgent Project Map

What we are building, what is real today, and what we deliver in November

AgriAgent Final Year Project · 11 October 2026

> **Read this first.** This is the single source of truth for the project. If another document, a code comment or an AI agent says something different, this document wins until the team agrees to change it. The Week 1 and Week 2 dossiers record history; this map describes the plan. To learn the project, start with `docs/guides/00_Start_Here.pdf`.

1 The project in one paragraph

A farmer's field contains a crop (for example cotton) and weeds. Spraying herbicide over the whole field wastes chemical and harms soil. **AgriAgent looks at a photo of a field, finds the weeds, and produces a map of only the weed areas so that only those areas would be sprayed.** The crop must never appear on the spray map. When the system is unsure what a plant is, it does not guess: it asks a slower reasoning model, and when that model recommends a herbicide, the recommendation must come from official Indian agricultural guidance (ICAR / CIBRC), not from the model's imagination.

AgriAgent is a **research prototype**. It produces a map on a screen and a GeoJSON file. It does not drive a real sprayer, and we will not claim it is field-ready.

2 The idea map

The system has five stages. Each stage is one idea you should be able to explain in an exam.

1. Detect: Grounding DINO
Reads the image and a text prompt such as "weed . cotton plant ." and draws boxes around things matching the words. Open-vocabulary: no training needed.

↓

2. Segment: SAM 2
Turns each box into a pixel-exact outline (mask). A refinement loop re-asks SAM 2 with a corrective point when a mask looks poor.

↓

3. Gate: DeterministicRouter (plain Python rules)
Keeps only masks whose label is a weed, removes anything that looks like crop, and suppresses the whole map if infestation is above 85%. This is the safety authority.

↓

4. Escalate (only when unsure): agronomy reasoning
Clef Flash flags ambiguity; a vision-language model (Qwen) diagnoses the weed; a knowledge store of ICAR/CIBRC documents supplies a legal herbicide and dose.

↓

5. Output: Streamlit dashboard + GeoJSON
Shows the boxes, masks, spray map, herbicide savings and prescription. Exports only approved weed zones.

The one rule that makes this project defensible: an AI model can advise, but only deterministic, testable code decides what goes on the spray map. Models can be wrong; the gate is what we can prove.

3 Honest status today (11 October 2026)

Every claim in this project must carry one of these labels.

Label	Meaning
REAL	Actually executed with real model weights on real images. Evidence exists.
MOCK	Executed with fake model outputs to test that the code's plumbing works. Says nothing about accuracy.
PARTIAL	Code exists and is tested, but has not run end to end with real models.
PLANNED	Not built yet.
BLOCKED	Cannot run with our current hardware or access.

Component	Status	What that means
Grounding DINO + SAM 2	REAL once	Ran once on a Kaggle T4 GPU (3 October). The output was wrong; see Section 4.
Safety gate + GeoJSON export	PARTIAL	Tested with fake masks (175 unit + 35 end-to-end tests pass, CI green). Never tested on real masks.
Streamlit app	MOCK	Starts in CI. Nobody on the team has opened it and used it yet.
Agronomy (VLM + RAG + prescription)	MOCK	Logic exists and is tested offline. Never called a real model.
Groq `qwen/qwen3.8-27b`	PARTIAL	Client code exists. Needs an API key. Never called from the pipeline. Accepts images (checked 12 Oct). Marked Preview on Groq.
Local `Qwen3.5-4B` (llama-server)	PARTIAL	Client code exists (OpenAI-compatible). Runs as a 4-bit GGUF with vision file in about 3.5 GB; team has run Qwen3.5 4B/9B GGUFs before.
Clef Flash (Cloudflare or local)	PLANNED	**No longer blocked** (12 Oct): hosted free on Cloudflare Workers AI with images; locally as a 6.5 GB GGUF through llama.cpp (text only). Adapter must be rewritten (schedule task T2.8).
Evaluation numbers	MOCK	All previous metric files had `mock_mode: true`. We have no real accuracy numbers yet.

4 The most important finding so far

The single real run (Kaggle T4, image `docs/images/live_real_spray_map_kaggle.png`) was a cotton field with weeds. Grounding DINO returned **one box labelled "vegetation" covering the whole field**, and SAM 2 then marked **about 79% of the image, cotton included,** as spray area.

> **What this teaches us.** The hard problem in this project is not drawing masks. It is **telling weed from crop**. A generic prompt like "weed" or "vegetation" does not separate them. Everything in the remaining weeks should serve that question: *How much crop ends up on the spray map, and how much weed is missed?*

This is a good result for a research project, because it is honest and it points at the real work.

5 What we deliver (second week of November)

These five items are the final deliverable. Anything not on this list is optional.

1. **A working demo.** Streamlit app on our own laptop: upload a real field photo, see boxes, masks, the spray map and the gate's decision. Vision models run locally on the RTX 5050; reasoning uses Groq. A recorded demo video as backup.
2. **Real evaluation.** Run the pipeline on a fixed set of annotated real images (CottonWeedDet3 and/or Sugar Beets). Report, at minimum:
   - weed detection quality (precision / recall, mask IoU against ground truth);
   - **crop protection**: percentage of crop pixels that ended up on the spray map (target: as close to 0 as possible);
   - herbicide savings versus spraying the whole image.

   Every number states the model version, the device, and that it is a real run.
3. **Weed-vs-crop improvement.** At least one change that measurably reduces crop on the spray map compared with the baseline (for example: prompting for both "cotton plant" and "weed" so crop boxes are detected and excluded, or asking the VLM to classify uncertain boxes). Before/after numbers.
4. **Safety design + tests.** The deterministic gate, the approved-mask-only export, and the tests that prove them. Already mostly done.
5. **Report and viva.** Final report written in Typst, a slide deck, and every team member able to explain the full pipeline and their own track.

6 Model decisions (frozen)

Changing models repeatedly is what confused this project. These choices are now frozen.

Role	Model	Decision
Detection	`IDEA-Research/grounding-dino-tiny`	Core. Runs locally.
Segmentation	`facebook/sam2-hiera-tiny`	Core. Runs locally.
Reasoning (cloud)	Groq `qwen/qwen3.8-27b`	Primary for diagnosis and prescription text. Backup: Groq `openai/gpt-oss-120b` (text only).
Reasoning (local)	`Qwen3.5-4B` GGUF via `llama-server`	Offline backup, and one of the three candidates for the crop-or-weed image check.
Ambiguity triage	Clef Flash: Cloudflare Workers AI `@cf/cloudflare/clef-flash`	Primary (free tier, images). Backup: local GGUF via `llama-server` (text only). Advisory only.

Updated 12 October: Clef Flash moved from "not runnable" to the triage model, and the crop-or-weed check compares three back-ends; details and setup steps in `docs/AgriAgent_Schedule.pdf` (Section 3 and Appendix A). No other model is added before the deadline unless the whole team agrees in writing.

7 Our hardware

Measured on 11 October 2026: Intel i5-13450HX (16 threads), 15 GB RAM, NVIDIA RTX 5050 Laptop GPU with **8 GB VRAM**, about 33 GB free disk. Grounding DINO-tiny and SAM 2-tiny together fit comfortably. A 4-bit Qwen3.5-4B (about 3.5 GB with its vision file) fits beside them. A 4-bit Clef Flash (6.5 GB) fits only on its own, so it runs after the vision models, or hosted on Cloudflare. Kaggle (free T4, 16 GB) remains available for heavier runs.

8 Team tracks

Names follow the Git branches. Each track owns one part of the deliverable.

Track	Owns	Delivers by November
1 · Chaitanya Core vision + safety	`src/vision/`, `src/workflow.py`, `src/agents/`	First real local run; weed-vs-crop improvement; gate design fixes (Section 9).
2 · Sumant Agronomy	`src/agronomy/`	All model back-ends (Groq, Cloudflare Clef Flash, local llama-server); real Groq call end to end; prescriptions checked against the ICAR bulletin for 5 weeds.
3 · Sahil Data + evaluation	`data/`, `src/evaluation/`	Fixed real evaluation set with annotations; real metrics table incl. crop protection.
4 · Amit UI + integration	`app/`, `.github/`, `tests/e2e/`	Demo-ready Streamlit app on real images; demo video; CI stays green.

9 Known design issues to fix

Found while writing this map. They are the core technical tasks for Track 1.

1. **The gate ignores per-mask confidence.** In `spray_map_node` (`src/workflow.py`), the gate is called with confidence and IoU hard-coded to `1.0`, so only the 85% infestation rule really applies. A low-confidence weed mask still reaches the spray map.
2. **Escalation happens after the map is approved.** `triage_node` runs after `spray_map_node`. If triage says "unsure, escalate", the spray map has already been approved and is not withdrawn.
3. **Labels come from the prompt, not from understanding.** A mask is "weed" because the prompt phrase "weed" matched, not because the model distinguished weed from crop. This is the Section 4 problem.
4. **Prompt and gate disagree.** The default prompt in `scripts/run_live_pipeline.py` is `"cotton weed . broadleaf plant ."`. The gate treats any label containing "cotton" as crop and "plant" as non-weed, so with this prompt nothing would ever be sprayed. Prompt and label policy must be designed together.
5. **More issues found while writing the study guides.** Silent fallback to mock when weights fail to load; mock refinement fakes an IoU gain; the app's confidence slider does not affect the spray map; hard-coded fallback herbicide in the GeoJSON; savings differ with and without OpenCV; statutory tables unchecked against the PDF. Full list with line numbers: Guide 2, Section 10.
6. **Unverified speed claims.** Code comments mention "< 40 ms" latency. Nothing has measured this. Treat as a target, not a fact.

10 Timeline

The detailed week-by-week plan, with every member's tasks, testing roles and checkpoints, is in `docs/AgriAgent_Schedule.pdf`. Summary:

When	Goal
12–14 Oct	Repo frozen. Everyone studies this map and their own track's code.
15–18 Oct	First real local run of DINO + SAM 2 on the laptop. Evaluation annotations audited. Test plans written.
19–25 Oct	Evaluation set frozen (21 Oct). Baseline real metrics incl. crop on the spray map. Gate issues 1, 4, 5 fixed. Groq live.
26 Oct – 1 Nov	Weed-vs-crop improvement measured. Gate issue 2 fixed. Test cycle 1.
2–8 Nov	Defect fixes, regression cycle 2, code freeze on **Thu 5 Nov**, demo video, full report draft. Diwali weekend off.
9–13 Nov	Final report and slides, mock viva, submission and viva.

11 Rules for working with AI coding agents

These rules exist because uncontrolled agents caused most of the confusion so far.

- One agent at a time in a working copy. Never two agents editing the same repo simultaneously.
- Agents must not write status claims ("complete", "verified", "100%") unless a command was run and its output is pasted as evidence.
- Agents must not change model choices or add dependencies without a team decision.
- No new documents. Update this map, the schedule, the README, or the current week's dossier.
- Every change goes through a pull request that a human on the team has read and understood.

12 Commands

```
# setup (Python 3.10 or 3.11)
python -m venv .venv && source .venv/bin/activate
python -m pip install -e '.[dev]'

# all checks (same as CI)
ruff check . && ruff format --check .
mypy --config-file pyproject.toml src/
PYTHONPATH=. python -m unittest discover tests
PYTHONPATH=. python tests/e2e/runner.py --all

# run the app
streamlit run app/app.py

# real pipeline on one image (needs model weights)
python scripts/run_live_pipeline.py --image path/to/field.jpg
```